E-commerce-Sales-Analysis-Pipeline

Anil Saini · 21 April 2026

E-commerce-Sales-Analysis-Pipeline

AUTHOR: Anil Saini

DATE: March 2026

TOOLS: MySQL, Python, Power BI

DATASET: E-commerce Dataset (Kaggle)

PURPOSE: Identify Negative Profit Through End-to-End ETL Pipeline

In [ ]:

```
# --- STEP 1: DEPENDENCIES & AUTHENTICATION ---
import pandas as pd
import getpass
from sqlalchemy import create_engine
import matplotlib.pyplot as plt
import seaborn as sns

# Securely capture credentials
db_password = getpass.getpass("Enter Database Password: ")
```

In [ ]:

```
# --- STEP 2: DATA EXTRACTION ---
# Database configuration
connection_url = f'mysql+pymysql://root:{db_password}@localhost:3306/project'
db_engine = create_engine(connection_url)

# SQL Query with formatted JOIN logic
query = """
SELECT * FROM `list of orders`
INNER JOIN `order details`
    ON `list of orders`.`Order ID` = `order details`.`Order ID`
"""

# Load raw data and handle duplicate columns from the JOIN
raw_df = pd.read_sql_query(query, db_engine)
df = raw_df.loc[:, ~raw_df.columns.duplicated()].copy()
```

In [ ]:

```
# --- STEP 3: DATA AUDIT (Exploratory Data Analysis) ---
print("\n--- [Data Audit: Info] ---")
df.info()

print("\n--- [Data Audit: Statistics] ---")
print(df.describe())

print("\n--- [Data Audit: Missing Values] ---")
print(df.isnull().sum())
```

Out [ ]:

```
--- [Data Audit: Info] ---
<class 'pandas.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Order ID      1500 non-null   str
 1   Order Date    1500 non-null   str
 2   CustomerName  1500 non-null   str
 3   State         1500 non-null   str
 4   City          1500 non-null   str
 5   Amount        1500 non-null   float64
 6   Profit        1500 non-null   float64
```

```
 7   Quantity      1500 non-null   int64
 8   Category      1500 non-null   str
 9   Sub-Category  1500 non-null   str
dtypes: float64(2), int64(1), str(7)
memory usage: 117.3 KB

--- [Data Audit: Statistics] ---
            Amount       Profit     Quantity
count  1500.000000  1500.000000  1500.000000
mean    287.668000    15.970000     3.743333
std     461.050488   169.140565     2.184942
min       4.000000 -1981.000000     1.000000
25%      45.000000    -9.250000     2.000000
50%     118.000000     9.000000     3.000000
75%     322.000000    38.000000     5.000000
max    5729.000000  1698.000000    14.000000

--- [Data Audit: Missing Values] ---
Order ID        0
Order Date      0
CustomerName    0
State           0
City            0
Amount          0
Profit          0
Quantity        0
Category        0
Sub-Category    0
dtype: int64
```

In [ ]:

```python
# --- STEP 4: TRANSFORMATION & FINAL VERIFICATION ---
# Standardizing Date Format
df['Order Date'] = pd.to_datetime(df['Order Date'], dayfirst=True)

# Extracting Temporal Features (Time-series components)
df['Order Day'] = df['Order Date'].dt.day_name()
df['Order Month'] = df['Order Date'].dt.month_name()
df['Order Year'] = df['Order Date'].dt.year

# Calculating Financial KPI: Profit Margin Percentage
df['Profit Margin'] = (df['Profit'] / df['Amount']) * 100

print("\n[Success] Transformation Complete. Final Dataset Preview:")
print(df.head())
```

Out [ ]:

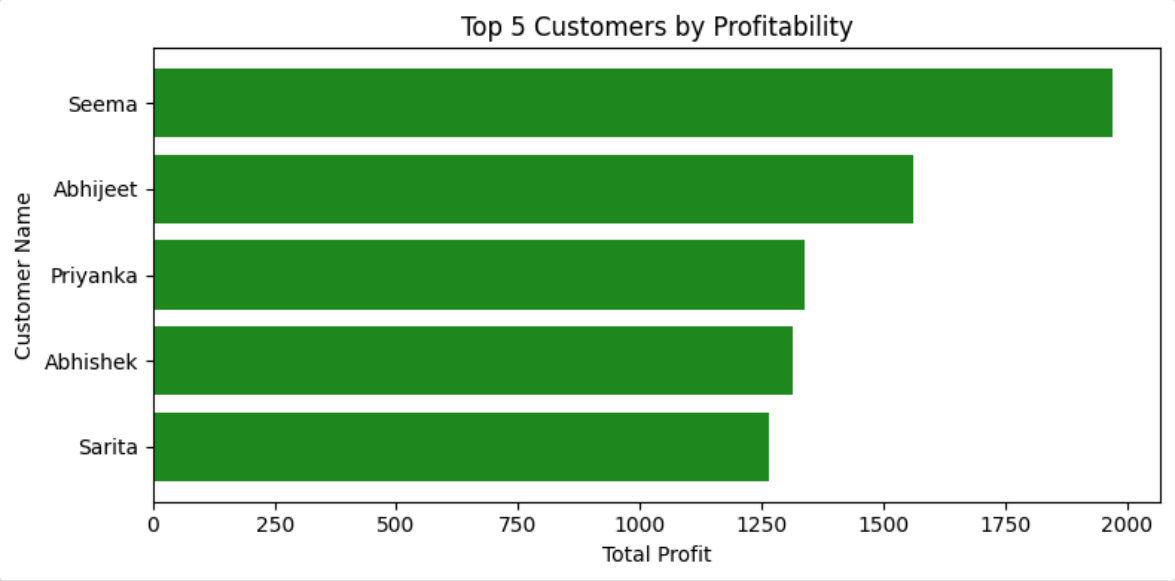
```
[Success] Transformation Complete. Final Dataset Preview:
  Order ID Order Date CustomerName        State       City  Amount  Profit  \
0  B-25601 2018-04-01       Bharat      Gujarat  Ahmedabad  1275.0 -1148.0
1  B-25601 2018-04-01       Bharat      Gujarat  Ahmedabad    66.0   -12.0
2  B-25601 2018-04-01       Bharat      Gujarat  Ahmedabad     8.0    -2.0
3  B-25601 2018-04-01       Bharat      Gujarat  Ahmedabad    80.0   -56.0
4  B-25602 2018-04-01        Pearl  Maharashtra       Pune   168.0  -111.0

   Quantity     Category      Sub-Category Order Day Order Month  Order Year  \
0         7    Furniture         Bookcases    Sunday       April        2018
1         5     Clothing             Stole    Sunday       April        2018
2         3     Clothing       Hankerchief    Sunday       April        2018
3         4  Electronics  Electronic Games    Sunday       April        2018
4         2  Electronics            Phones    Sunday       April        2018

   Profit Margin
0     -90.039216
1     -18.181818
2     -25.000000
3     -70.000000
4     -66.071429
```

In [14]:

```python
# --- STEP 5: TOP PERFORMING CUSTOMERS ---
# Identifying the Top 5 customers by total profit
top_customers = df.groupby('CustomerName')['Profit'].sum().sort_values(ascending=False).head(5)
```

```
plt.figure(figsize=(8, 4))
plt.barh(top_customers.index, top_customers.values, color='forestgreen')
plt.gca().invert_yaxis()  # Ensures the top performer is at the top of the chart
plt.title('Top 5 Customers by Profitability')
plt.xlabel('Total Profit')
plt.ylabel('Customer Name')
plt.tight_layout()
plt.show()
```

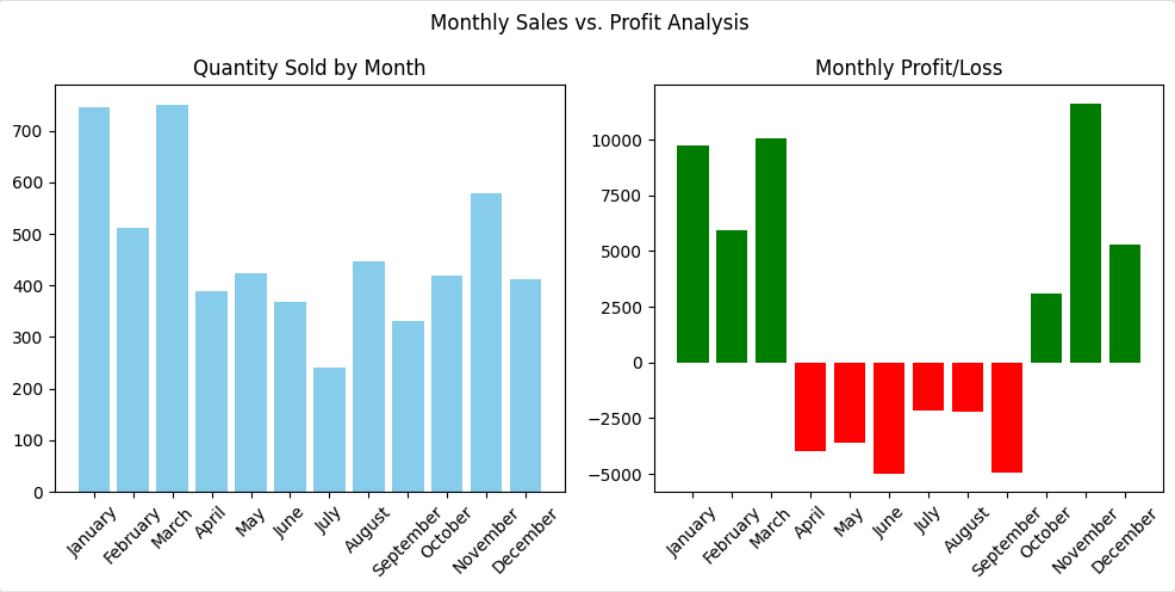
Out [14]:



In [16]:

```
# --- STEP 6: MONTH-WISE PERFORMANCE TRENDS ---
months_order = ['January', 'February', 'March', 'April', 'May', 'June',
                'July', 'August', 'September', 'October', 'November', 'December']
df['Order Month'] = pd.Categorical(df['Order Month'], categories=months_order, ordered=True)

monthly_qty = df.groupby('Order Month', observed=False)['Quantity'].sum()
monthly_profit = df.groupby('Order Month', observed=False)['Profit'].sum()

plt.figure(figsize=(10, 5))
plt.suptitle("Monthly Sales vs. Profit Analysis")

plt.subplot(1, 2, 1)
plt.bar(monthly_qty.index, monthly_qty.values, color='skyblue')
plt.title('Quantity Sold by Month')
plt.xticks(rotation=45)

plt.subplot(1, 2, 2)
# Green for profit, Red for loss
plt.bar(monthly_profit.index, monthly_profit.values, color=['green' if x > 0 else 'red' for x in monthly_profit.values])
plt.title('Monthly Profit/Loss')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

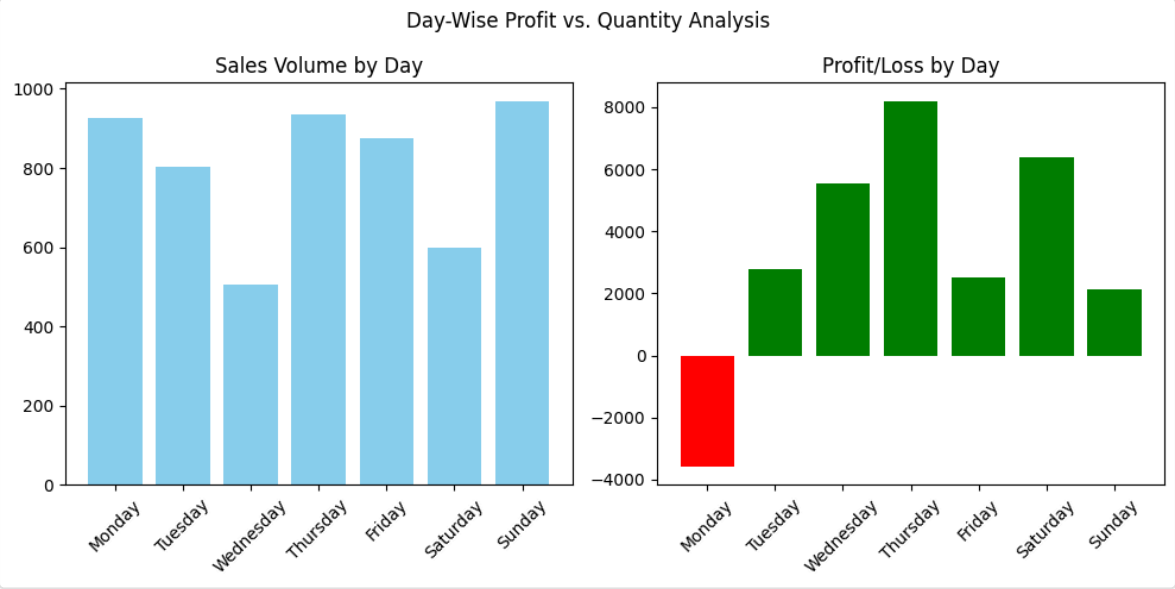
Out [16]:



In [17]:

```
# --- STEP 7: DAY-WISE (WEEKLY) TRENDS ---
days_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
df['Order Day'] = pd.Categorical(df['Order Day'], categories=days_order, ordered=True)

daily_qty = df.groupby('Order Day', observed=False)['Quantity'].sum()
daily_profit = df.groupby('Order Day', observed=False)['Profit'].sum()

plt.figure(figsize=(10, 5))
plt.suptitle("Day-Wise Profit vs. Quantity Analysis")

plt.subplot(1, 2, 1)
plt.bar(daily_qty.index, daily_qty.values, color='skyblue')
plt.title('Sales Volume by Day')
plt.xticks(rotation=45)

plt.subplot(1, 2, 2)
plt.bar(daily_profit.index, daily_profit.values, color=['green' if x > 0 else 'red' for x in daily_profit.values])
plt.title('Profit/Loss by Day')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

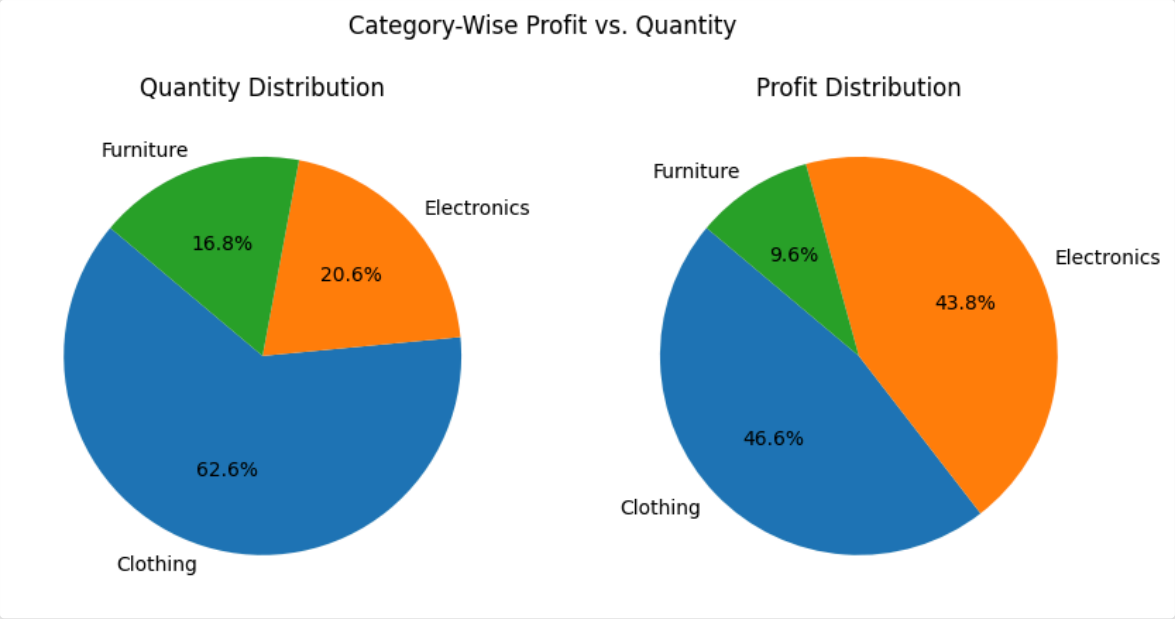
Out [17]:



In [ ]:

```
# --- STEP 8: CATEGORY DISTRIBUTION (MARKET SHARE) ---
cat_qty = df.groupby('Category')['Quantity'].sum()
cat_profit = df.groupby('Category')['Profit'].sum()

plt.figure(figsize=(10, 5))
plt.suptitle('Category-Wise Profit vs. Quantity')

plt.subplot(1, 2, 1)
plt.pie(cat_qty, labels=cat_qty.index, autopct='%1.1f%%', startangle=140)
plt.title('Quantity Distribution')

plt.subplot(1, 2, 2)
plt.pie(cat_profit, labels=cat_profit.index, autopct='%1.1f%%', startangle=140)
plt.title('Profit Distribution')
plt.show()
```

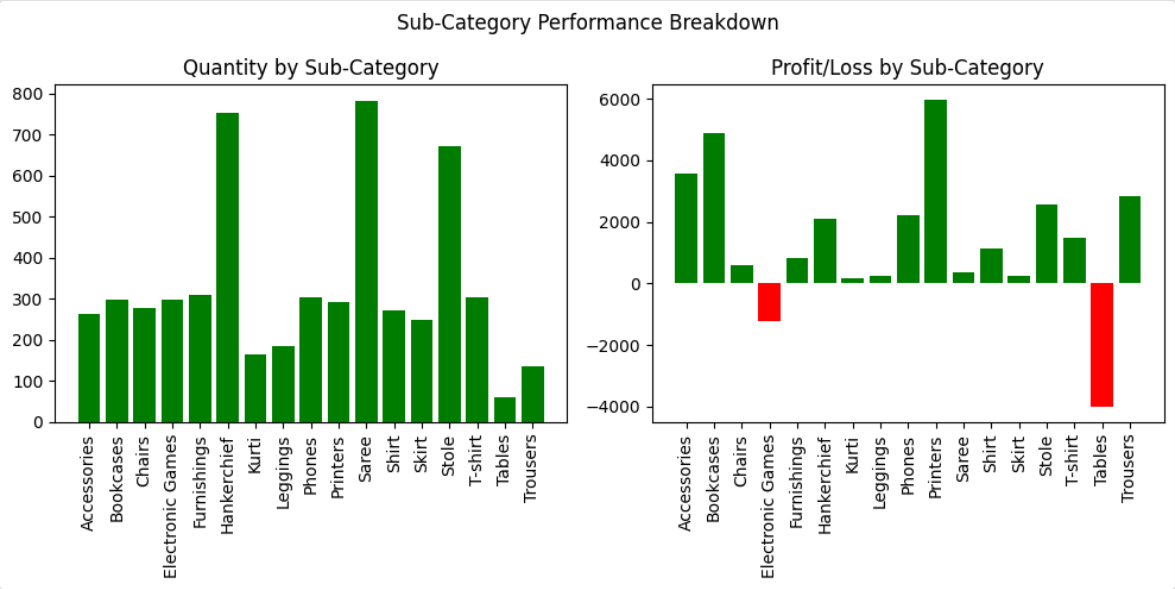
Out [ ]:



In [ ]:

```
# --- STEP 9: SUB-CATEGORY GRANULAR ANALYSIS ---
sub_qty = df.groupby('Sub-Category')['Quantity'].sum()
sub_profit = df.groupby('Sub-Category')['Profit'].sum()

plt.figure(figsize=(10, 5))
plt.suptitle("Sub-Category Performance Breakdown")

plt.subplot(1, 2, 1)
plt.bar(sub_qty.index, sub_qty.values, color='green')
plt.title('Quantity by Sub-Category')
plt.xticks(rotation=90)

plt.subplot(1, 2, 2)
plt.bar(sub_profit.index, sub_profit.values, color=['green' if x > 0 else 'red' for x in sub_profit.values])
plt.title('Profit/Loss by Sub-Category')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

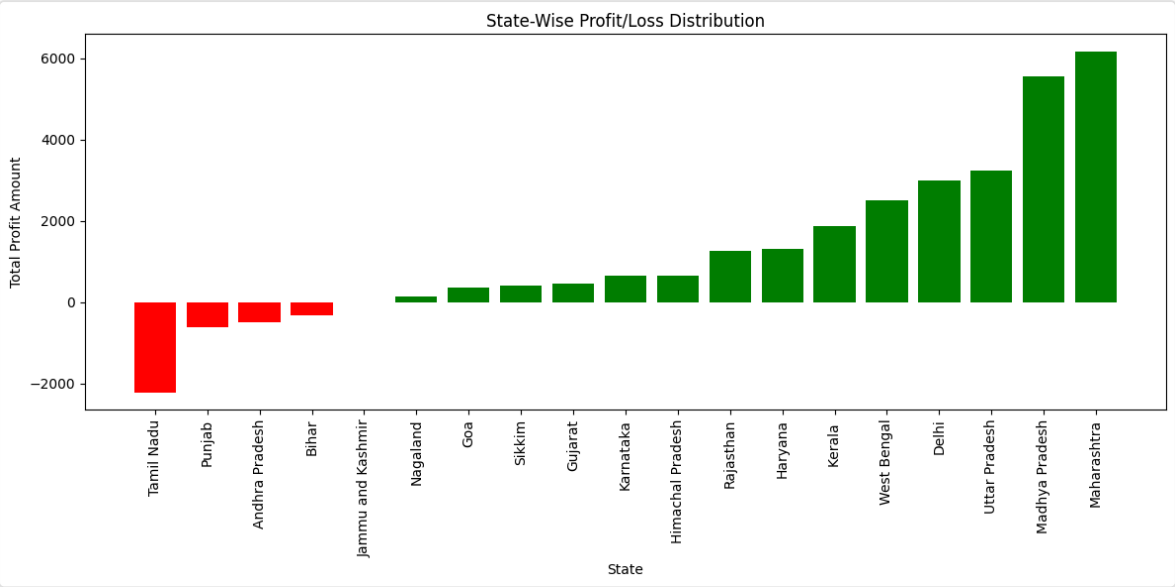
Out [ ]:



In [ ]:

```
# --- STEP 10: GEOGRAPHICAL PROFITABILITY ---
state_profit = df.groupby('State')['Profit'].sum().sort_values()

plt.figure(figsize=(12, 6))
plt.bar(state_profit.index, state_profit.values, color=['green' if x > 0 else 'red' for x in state_profit.values])
plt.xticks(rotation=90)
plt.title("State-Wise Profit/Loss Distribution")
plt.xlabel("State")
plt.ylabel("Total Profit Amount")
plt.tight_layout()
plt.show()
```

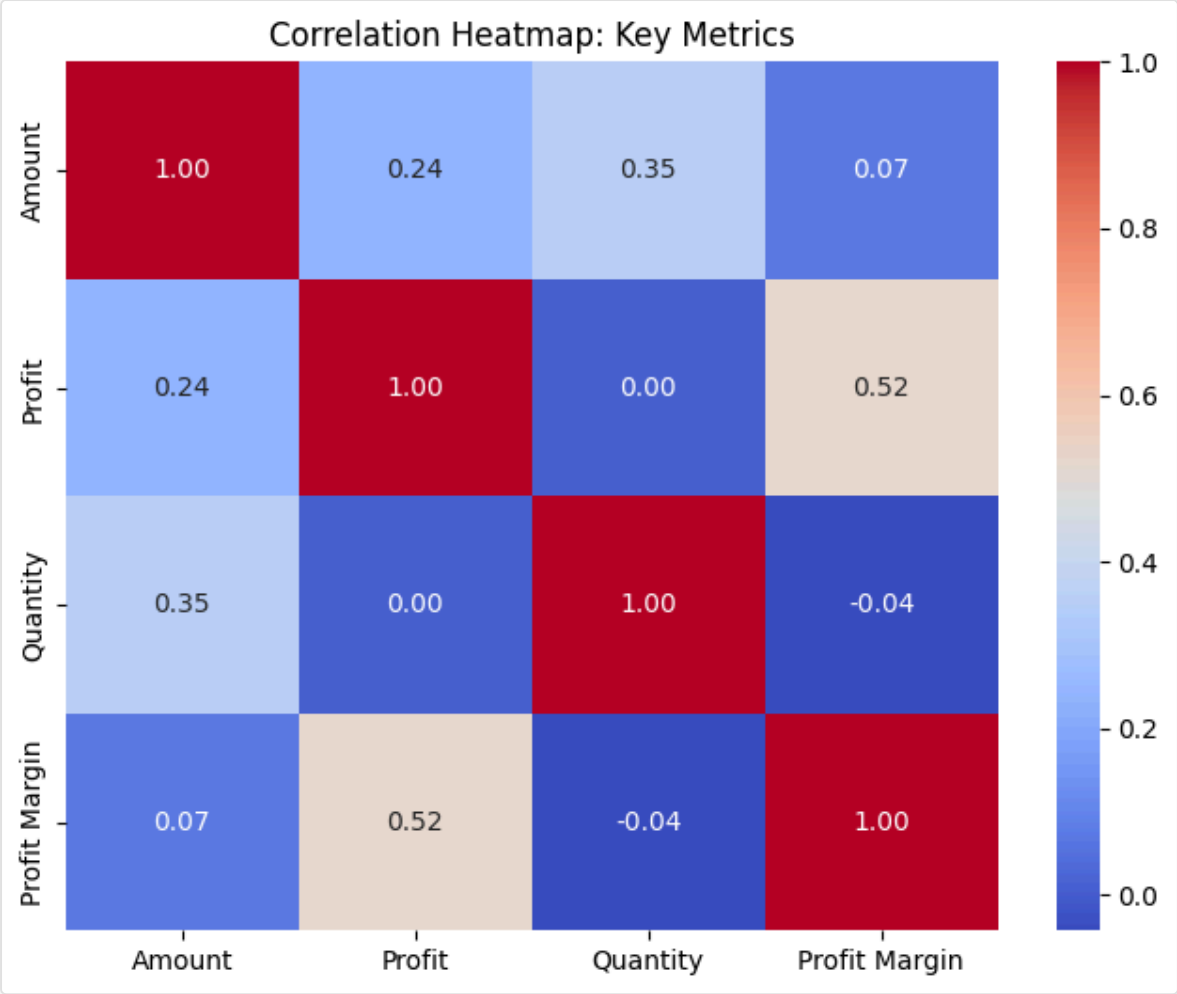
Out [ ]:



In [22]:

```
# --- STEP 11: STATISTICAL CORRELATION ---
plt.figure(figsize=(8, 6))
correlation_matrix = df[['Amount', 'Profit', 'Quantity', 'Profit Margin']].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap: Key Metrics')
plt.show()
```

Out [22]:


Correlation Heatmap: Key Metrics
Amount
Profit
Quantity
Profit Margin
1.00
0.24
0.35
0.07
0.24
1.00
0.00
0.52
0.35
0.00
1.00
-0.04
0.07
0.52
-0.04
1.00
Amount
Profit
Quantity
Profit Margin
1.0
0.8
0.6
0.4
0.2
0.0